Bioinformática 2025

TP Nº5: Manejo y acceso de estructuras.

El objetivo de esta práctica es utilizar biopython y lo aprendido de programación para manipular estructuras y determinar propiedades estructurales.

Objetivo 1) Cargar y representar estructuras en Biopython.

El punto inicial (como en secuencia) consiste en utilizar Biopython para recorrer un archivo PDB que luego genera un objeto que contiene la estructura. Es importante destacar que desde 2014 el formato estándar para estructuras de biomoléculas (en el Protein Data bank) es el macromolecular Crystallographic Information File (mmCIF) file.

A) Parsear archivos mmCIF y PDB, navegarlos, y obtener información relevante relacionada con la estructura.

```
>>> from Bio.PDB.MMCIFParser import MMCIFParser
>>> parser = MMCIFParser()
>>> structure = parser.get_structure("MiCIF", "file.cif")
```

Incluso se puede levantar el mmcif en un diccionario (pre)establecido que permite algunas funciones adicionales.

```
>>> from Bio.PDB.MMCIF2Dict import MMCIF2Dict
>>> mmcif_dict = MMCIF2Dict("file.cif")
```

Para PARSEAR un pdb primero cargamos el parser (el PERMISSIVE=1 permite que el PDB no esté perfecto), luego definimos el id que tendrá ESA estructura en nuestro programa

```
>>> from Bio.PDB.PDBParser import PDBParser
>>> parser = PDBParser(PERMISSIVE=1)
>>> structure_id = 'miPDB'
>>> filename = ‘miPDB.pdb’
>>> structure = parser.get_structure(structure_id, filename)
```

Una vez cargada la estructura, podemos pedirle (de manera similar a lo que hicimos con la secuencia, varias de las propiedades)

Por ejemplo:

```
>>> resolution = structure.header["resolution"]
```

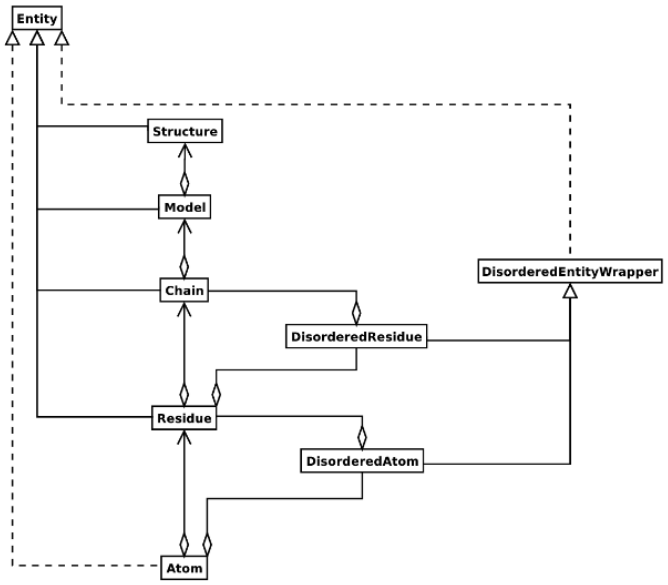
The available keys (recuerde que son las llaves del diccionario) are:

name, head, deposition_date, release_date, structure_method, resolution, structure_reference (which maps to a list of references), journal_reference, author, compound (which maps to a dictionary with various information about the crystallized compound), has_missing_residues, and missing_residues.

El modulo también permite grabar archivos PDB/mmcif (pero a esta altura creemos que ud puede entenderlo solo mirando el tutorial de biopython)

En biopython la representaciòn “interna” de la estructura es la que corresponde al modelo SMCRA (Structure/Model/Chain/Residue/Atom) architecture:

- A structure consists of models
- A model consists of chains
- A chain consists of residues
- A residue consists of atoms



Y es la manera en que se “recorren” las estructuras con los loops.
Además, cada RESIDUO se accede utilizando el “full_id” una tupla que contiene los siguientes

ítems:
("estructura", Modelo, "cadena", (residuo))
Estructura es un String con: structure_id
Modelo es un entero empezando en 0
Cadena, las cadenas A, B, C etc

Entonces para elegir un residuo particular uso:

```
>>> model = structure[0]
>>> chain = model['A']
>>> residue = chain[100]
```

Aca estoy eligiendo el residuo 100 , de la cadena “A” del modelo de la estructura
A un residuo -ya definido- además se le pueden preguntar cosas:

```
>>> residue.get_resname()    # returns the residue name, e.g. "ASN"
>>> residue.is_disordered()# returns 1 if the residue has disordered
atoms
>>> residue.get_segid()      # returns the SEGID, e.g. "CHN1"
```

Nota: Residuo es en realidad una tupla de 3 elementos:
The **hetero-field** (hetfield) es:
'W' in the case of a water molecule
'H_' followed by the residue name for other hetero residues (e.g. 'H_GLC')
blank for standard amino and nucleic acids
The **sequence identifier** (resseq), un entero que es el Nro de reisudo (e.g., 100);
The **insertion code** (icode); a string, e.g. ’A’.

Si queremos saber la tupla “id” del residuo (residue.id)
y si queremos toda la informacion (residue_full.id):

```
>>> print(residue.id)
>>> print(residue.full_id)
```

ÀTOMO
El pròximo objeto importante cómo se imagina es el **átomo**. El id de un àtomo es, por supuesto, su nombre, ejemplo “OG” u “CA”:

Entonces, siguiendo la estructura SMRCA, si queremos obtener un àtomo en particular (usando las propiedades de SMCRA

```
>>> atom = structure[0]["A"][100]["CA"]
>>> print(atom.full_id)
```

El àtomo posee las siguientes propiedades (entre otras):

```
>>> a.get_name()       # atom name (spaces stripped, e.g. "CA")
>>> a.get_id()         # id (equals atom name)
>>> a.get_coord()      # atomic coordinates
>>> a.get_vector()     # atomic coordinates as Vector object
>>> a.get_bfactor()    # isotropic B factor
>>> a.get_occupancy()  # occupancy
>>> a.get_altloc()     # alternative location specifier
>>> a.get_sigatm()     # standard deviation of atomic parameters
>>> a.get_siguij()     # standard deviation of anisotropic B
factor
>>> a.get_anisou()     # anisotropic B factor
>>> a.get_fullname()   # atom name (with spaces, e.g. ".CA.")
```

Recorriendo la estructura

Para recorrer las estructuras se utilizan los típicos loops de tipo for:

Por ejemplo Iteremos entonces sobre todos los residuos:

```
residues = model.get_residues()
for residue in residues:
    print(residue)
```

O sobre todos los átomos:

```
for model in structure:
    for chain in model:
        for residue in chain:
            for atom in residue:
                print(atom)
```

O mas corto:

```
atoms = structure.get_atoms()
    for atom in atoms:
        print(atom)
```

Objetivo 2) Instalar y probar programa de visualiziaciòn de estructuras

Utilizaremos NGLVIEW (http://nglviewer.org/#nglview)
Primero desde la consola:

```
pyhton3  -m pip install nglview
```

Después hay que activarlo:

```
jupyter-nbextension enable nglview --py --sys-prefix
```

Ahora vamos a visualizar una estructura utilizando nglview

```
import nglview as nv
structure = parser.get_structure(structure_id, pdb_id)
view = nv.show_biopython(structure)
view
```

De esta manera, obtenemos la representación por defecto que nos ofrece nglview. Pero también podemos pedirle representaciones específicas.

```
view2 = nv.show_biopython(structure)
view2.clear_representations()
view2.add_representation('cartoon',
selection='protein', color='blue')
view2.add_representation('licorice', selection='not
hydrogen')
view2
```

Si tiene problemas instalando nglview, otra opción para visualizar estructuras que funciona desde colab es py3Dmol, a continuación le dejamos un link con un módulo de tutorial sobre como utilizarlo y posibles aplicaciones:

Mini_Tutorial_py3Dmol.ipynb

Objetivo 3) Ahora vamos a calcular algunas propiedades del PDB

Para medir una distancia, simplemente obtenemos 2 átomos a1 y a2 y hacemos la resta:

```
a1 = structure[0]["A"][100]["CA"]
a2 = structure[0]["A"][101]["CA"]
distance = a1-a2
```

Para medir un “ángulo” necesitamos la representación vectorial de los tres atomos involucrados

```
vector1 = atom1.get_vector()
vector2 = atom2.get_vector()
vector3 = atom3.get_vector()
angle = calc_angle(vector1, vector2, vector3)
```

Y si son 4 podemos medir el diedro…

```
angle = calc_dihedral(vector1, vector2, vector3, vector4)
```

Objetivo 4) Superponer estructuras

Una de las tareas más importantes en el manejo de estructuras es lo que comúnmente llamamos alineamiento-estructural que en realidad consiste en superponer estructuras.
Para superponer dos estructuras hay que comprender lo siguiente:
Hay una estructura que es la que denominamos REFERENCIA (esta es la que NO se va a mover) y una segunda estructura que será la MÓVIL
Primero se definen dos conjuntos “átomos” que deben contener ambas estructuras REF y MÓVIL que es el que se usa para calcular la matriz de transición/rotation (Matrix). Ambos conjuntos deben tener igual dimensión (la idea es que sean los átomos equivalentes, por ejemplo los CA compartidos o un dado residuo o los aa de un sitio activo)
Una vez definida la matriz esta puede ser utilizada para mover toda la estructura MÓVIL

El módulo se utiliza asi:
#primero defino los sets de atomos sup_ref y sup_movil (usando lo que ya aprendimos)

```
#calculo la matriz
>>> sup = Superimposer()
>>> sup.set_atoms(sup_ref, sup_movil)

#Puedo imprimir la matriz y el RMSD
>>> print(sup.rotran)
>>> print(sup.rms)
# Aplico la matriz sobre la estructura MOVIL
>>> sup.apply(movil)
```

Ejercicio 1)

Baje del Protein Data bank una estructura de su elección tanto en formato PDB como en formato CIF (sino puede usar la que está en el directorio que corresponde a la Mioglobina). Utilice biopython para cargar ambos archivos y explorar un poco qué propiedades del header y demás puede obtener.

Ejercicio 2)

a) utilice los comandos anteriores para tratar de: identificar el residuo HEMO de la Mb y

obtener su id y full_id (hay algun otro residuo que NO es de la proteìna y NO es agua?)

b) Levante un PDB y obtenga: a) el número de cadenas, b) el número de residuos, c) número de átomos totales, d) número de átomos de proteìna, e) número de HETatoms, f) número de átomos de agua.

c) Levante un PDB y determine la secuencia de aminoàcidos de cada cadena

d) Levante un PDB y obtenga un nuevo PDB que contenga “solo” los CA

Ejercicio 3)

Juegue un poco con el nglview, intente gtraficar uno (o algunos pocos residuos aislados, cambiar el tipo de representación etc.)

Ejercicio 4)

Busque en el PDB la oximioglobina (si es valiente la oxihemoglobina o algo equivalente), carguela en Biopython y calcule: distancia Fe-O, Angulo Fe-O-O, Dihedro N-Fe-O-O

Ejercicio 5)

Elija dos proteìnas homólogas (por ejemplo la misma proteìna de dos organismos diferentes) obtenga las estructuras y superponerlas.

Ejercicio 6)

i) Medir la superficie accesible expuesta de los residuos utilizando la herramienta de Biopython Bio.PDB.SASA.

ii) Medir distancias entre CA, entre CB y entre los centros de masa de las cadenas laterales. En base a un valor de corte (puede ser 8A) definir los contactos.

iii) Evaluar la correlación entre el número de contactos medidos de diferentes formas y la SASA.

iv) Parece en el CB de las cadenas laterales, defina el radio de una esfera, y mida el número de vecinos polares (aminoácidos S,T,K,R,D,E,Q,N) y no polares (I,L,V,W,Y,F,M) midiendo distancias entre CA, entre CB, y desde el centro de masa de la cadena lateral (cm). Definir al menos 3 radios de esfera.

v) Evalúe la correlación entre el número de vecinos polares y no polares, y el pKa de los residuos de los PDB de la siguiente tabla.

PDB, Nro de residuo, pKa de referencia (aspartico), pKa experimental

1LNI, 17, 4, 3.72
1LNI, 25, 4, 4.87
1LNI, 33, 4, 2.39
1LNI, 79, 4, 7.37
1LNI, 84, 4, 3.01
1LNI, 93, 4, 3.09
1LSE, 18, 4, 2.66
1LSE, 48, 4, 1.6
1LSE, 52, 4, 3.68
1LSE, 66, 4, 0.9
1LSE, 87, 4, 2.07
1LSE, 101,4, 4.09
1LSE, 119, 4, 3.2

Ejercicio 7) Usando los modulos que armó previamente, lea el trabajo https://sci-hub.hllgczx.com/10.1016/j.jmb.2006.11.085 y repita el diseño experimental. Brevemente,

A) baje las estructuras mencionadas en formato PDB,

Considerando solo los carbonos alfa:

B) calcule los contactos para ambas estructuras usando el metodo de 'cutoff'.
C) calcule los contactos que estan SOLAMENTE en la estructura cerrada (1AKE), usando la definicion del trabajo. Calcule tambien los contactos exclusivos para cada estructura. Construya el mapa de contactos para cada estructura.
D) mida los angulos y los diedros considerando solamente los carbonos alfa. Note que en este caso, cada angulo normal y diedro esta compuesto de carbonos alfa, y que cada carbono alfa representa 1 residuo. Guarde en variables los angulos y los diedros.

Esta información la usaremos en el TP de dinamica molecular.

Ejercicio 8) Repita los cálculos para los PDB 1SU4, 4XOU y 3B9B. Haga lo mismo que en el punto 7 para cada par de estructuras.

Ejercicio 9) Implemente el calculo del mapa de contactos usando el algoritmo del trabajo https://sci-hub.hlgczx.com/10.1021/jp300852d. Considere solamente los carbonos alfa.

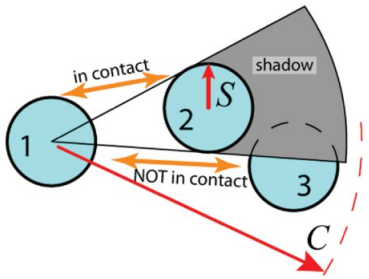


Figure 1. The Shadow contact map screening geometry. Only atoms within the cutoff distance *C* are considered. Atoms 1 and 2 are in contact because they are within *C* and have no intervening atom. To check if atoms 1 and 3 are in contact, one checks if atom 2 shadows atom 1 from atom 3. The three atoms are viewed in the plane, and all atoms are given the same shadowing radius *S*. Since a light shining from the center of atom 1 causes a shadow to be cast on atom 3, atoms 1 and 3 are not in contact.

Brevemente,

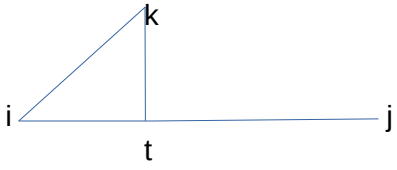
A) defina una distancia de contacto.
B) considere un radio S (radio de oclusion o sombra).
C) Encuentre los contactos.
D) Estudie los contactos del atomo i
E) Dado el contacto i con j (1 con 3 en la figura), trace una linea perpendicular desde los otros atomos (2 en la figura) en contacto con i al vector ij. Si la distancia del vector perpendicular k al vector ij < S, entonces ese atomo esta ocluyendo a j. Entonces ij no es un contacto real, pero ik sí lo es.

Consejo: use nociones de algebra entre vectores.

v_ij = vector entre i y j
k = posicion del atomo k

Objetivo: evaluar si la distancia entre un punto t perpendicular a ij saliendo desde k es menor a S. Si es menor a S, entonces hay oclusion.

Metodo: evaluar el punto t en la recta ij:



it = ((k − i) · v_ij) / (v_ij · v_ij), si t=0, entonces es i, si t=1, entonces es j, si t esta entre 0 y 1, entonces esta entre i y j.

Punto_t = i + t . v, distancia perpendicular entre punto_t y k = (k – p) * (k – p).
si punto_t < S entonces esta ocluyendo el contacto ij.

Sea p = i + t * v,

Grafique el mapa de contactos para diferentes valores de S.
Compare con el mapa de contactos definido por ‘cutoff’.